

Video 4: Advanced AI-Assisted Development

Deeper Control When Working with AI

Learning Objectives

By the end of this session, you should be able to:

- Explain context window limitations and their impact on AI-assisted development.
- Describe techniques for managing state using PROJECT_SPEC.md and chunking.
- Summarize the role of stubs in structuring incomplete code.
- Examine how duplication arises in AI-generated code and its risks.
- Recognize validation hooks as a mechanism for exposing assumptions and uncertainty.

Agenda

In this session, we'll discuss:

- Context Windows and State Management
- Incomplete Code Management with Stubs
- Duplication Detection and Removal
- Validation Hooks and Traceability

Context Windows and State Management

The AI can only see a finite window—and earlier details degrade

Understanding the Context Window Constraint

The AI has no persistent memory between sessions. Within a session, as the conversation grows, earlier details degrade or disappear entirely.

What Happens Without Management

- AI forgets requirements stated 20 prompts ago
- It contradicts decisions already agreed and built
- It generates duplicate logic because it can't see the full codebase
- Inconsistencies accumulate session after session
- Your codebase diverges from your original plan silently

Three Techniques to Manage Context

- Technique 1: Work in small, well-defined chunks per session
- Technique 2: Externalise state into PROJECT_SPEC.md
- Technique 3: Reset context and re-anchor at the start of new sessions

Manage context deliberately, not accidentally. The developers who do this have fewer surprises, fewer bugs, and faster progress.

Your Most Powerful Tool: The PROJECT_SPEC.md File

What to Put in Your PROJECT_SPEC.md

What you are building and why · Key components and their current status · Decisions already made · Constraints and rules the AI must follow · Current task and next task

How to Use It

Paste the relevant section at the start of every new session. "Here is our project context. We are currently working on Task 3. Do not modify Tasks 1 or 2."

When to Update It

After every session. Mark completed tasks. Record any new decisions. Add any constraints that emerged. Treat it as your source of truth.

What It Replaces

The mental model of "the AI remembers." It doesn't. This file is your AI's external memory—and yours too across long projects.

Your PROJECT_SPEC.md turns the AI from a forgetful collaborator into one that always starts with the full picture.

Managing Incomplete Code (Stubs)

Stubs are not a problem—they are a structured
to-do list

Reframing Stubs: From Frustration to Roadmap

When AI reaches uncertainty or practical limits, it fills gaps with stubs—placeholder functions marked with TODO. This is normal. The mistake is being surprised by it.

The Problem Approach

- Ask AI to generate everything in one go
- Receive code with hidden stubs and incomplete logic
- Run it—it fails in unexpected places
- Spend time hunting for what is incomplete

The Reframe

- Ask AI to scaffold with intentional stubs first
- Each stub = one task = one implementation prompt
- Clear TODO comments = visible, trackable progress
- Stubs become your sprint board

The Prompt to Use

"Before implementing, create a skeleton of the full application with all functions defined but not yet implemented. Use clear TODO comments explaining what each function should do and what inputs and outputs are expected."

A skeleton with clear TODOs is more valuable than rushed, half-working code. It is a map of your project.

Detecting and Managing Duplication

AI optimises locally—you must manage the global picture

Why AI Creates Duplication—and How to Fix It

AI generates code based on each prompt in isolation. It has no global view of your codebase. Over time this creates duplicate logic—the same function written twice, in different places, slightly differently.

Why Duplication Is Dangerous

- A bug fix in one copy doesn't fix the other
- The two copies may diverge over time, producing inconsistent behaviour
- Codebase becomes confusing—which version is the real one?
- Long-term instability that is hard to trace back to a source

Two Techniques to Control It

Technique 1: Periodic Refactoring Prompt (every 3-4 tasks):
"Review the code below for duplicated logic or functions. Suggest how to refactor it to eliminate duplication while keeping behaviour identical."

Technique 2: Inline Flags:

Add comments like: "NOTE: Similar logic exists in `validate_input()`—consider merging before next refactor."

**Never go more than 4 implementation tasks without running a duplication check.
Duplicate logic compounds faster than you expect.**

Validation Hooks and Traceability

Make the AI tell you what it might have gotten wrong

Embedding Validation Hooks in AI-Generated Code

A validation hook is a comment embedded in the code that makes assumptions, uncertainties, and incomplete sections visible—rather than hidden.

The Prompt That Creates Hooks

"After generating the code, add inline comments that identify: (1) any assumptions you made that I should verify, (2) any parts you are uncertain about, (3) any sections that are incomplete or require follow-up."

Example Output

```
# ASSUMPTION: Using 2024/25 UK brackets—verify these are correct  
# ASSUMPTION: No consideration for Scottish rates  
# TODO: Add handling for the personal savings allowance
```

What Validation Hooks Give You

- A map of where to review — not a search for problems
- Visible assumptions that you can verify or challenge
- Incomplete sections that become explicit tasks
- A structured inspection process instead of blind trust

Resolving Hooks Over Time

Verify → remove the ASSUMPTION comment. Complete → remove the TODO.
Clean code = resolved hooks.

Putting the Advanced Techniques Together

1

Start Fresh Session

Paste PROJECT_SPEC.md.
Re-anchor AI context. State
current task explicitly.



2

Scaffold with Stubs

Ask for skeleton first. All
functions defined, none yet
implemented. Review the
structure before building.



3

Implement with Hooks

Build one stub at a time. Ask
AI to annotate assumptions
and uncertainties after each
implementation.



4

Audit and Refactor

Every 3-4 tasks: run
duplication check, resolve
validation hooks, update
PROJECT_SPEC.md.

These four techniques together turn AI from a novelty into a reliable production tool for real Python projects.

They all address the same root problem: AI has no global awareness. These techniques give it one—artificially, but effectively.

Key Takeaways

01

Manage Context on Purpose

Use PROJECT_SPEC.md. Work in small session chunks. Reset and re-anchor regularly. Don't assume the AI remembers.

02

Stubs Are a Roadmap

Ask for intentional scaffolding. Each stub is a task. Each TODO is a checkpoint. Visible gaps are manageable gaps.

03

Audit and Validate Consistently

Run deduplication checks every 3-4 tasks. Use validation hooks to surface assumptions. Clean code means resolved hooks.

Next up: Video 5—Summary and Key Takeaways: The repeatable playbook and the honest costs of over-reliance

Summary

Here's a brief recap:

- Context windows define how AI retains and loses information across sessions.
- State management establishes external methods to maintain continuity and consistency.
- Stubs represent structured placeholders that guide incremental development.
- Duplication highlights the risk of locally generated code without global awareness.
- Validation hooks provide visibility into assumptions, uncertainties, and incomplete logic.

Learning Outcomes

You should now be able to:

- Manage AI context using PROJECT_SPEC.md, chunking, and session resets.
- Structure development workflows by creating and implementing stubs incrementally.
- Identify and refactor duplicated logic across AI-generated code.
- Embed and resolve validation hooks to verify assumptions and completeness.
- Maintain a consistent and traceable codebase across multiple AI-assisted sessions.